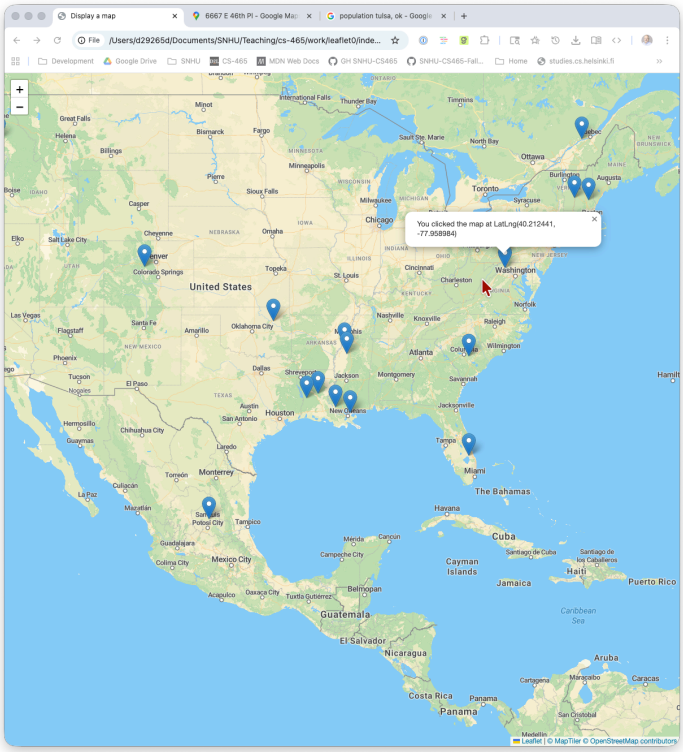
Lab 2: *Oh, the places you've been*

Create a single-page website to display a map and allow the user to click on the map to identify locations of interest to that user. It could be places they've lived, places they would like to visit, etc. You may choose to set the map view to be anywhere on Earth. You will use the Leaflet API and OpenStreetMap to create the map and add markers for each location.



Use React and Javascript to prompt the user to click on the map for each location. When the click is received, add a marker to the map and prompt the user for some information about that location. This information can be anything you want, e.g., years when they lived there, favorite restaurant there, etc. Think carefully about how large this area should be. After entering the information the user indicates they are finished by hitting `Esc` or `⏎ Enter`, which will save the entered information and close the information box.

Maintain a list of the places on the screen as they are entered. When the user clicks **Done**, stop prompting for locations, stop displaying the list of locations and just display the map with all the markers. If the user clicks on a marker, show the information they entered for that location until the user clicks on something else or hits `Esc` or `⏎ Enter` to close the information box.

Note: you should *tell* the user how to close the information box when you open it.

Finally, when the user clicks **Reset**, clear the map and the list of locations and start over.

Base requirements

1. Accept the assignment from the `Classroom50` link provided in BrightSpace. Use `git clone` to download a copy of the starter repository that is created for you. Use this repository to do your work and submit it for grading.

   > ⚠ **Warning**
   >
   > Any repository not created by Classroom50 cannot be submitted and will not be graded.

2. Provide the functionality specified in the previous section. Use the `vite` tool to build your solution (introduced in class on Friday, October 2, 2026).

3. **`README.md`**
   Update the provided `README.md` in the repository to include:

   - Your name after "Lab2 - CS465 Fall 2026" in the **`Author`** heading. This line is provided in the starter code `README.md` file.
   - A full description of your solution in the **`About this lab`** section
   - List any prerequisite software and versions that your solution was built with in the ***`Prerequisites`*** section. I should be able to start in an empty directory and, after installing these prerequisites, be able to build and run your system.
   - Explain how to build your solution in the **`Building this lab`** section.

     > **Important**
     >
     > If you do not provide complete instructions in the `README.md` file for me to build your Lab 2 submission, I will be unable to grade it. 😠

   - Explain how to run your solution, both locally and using the `onrender.com` URL in the **`How to run this lab`** section. Are there any other differences between how it runs locally and on the Internet?
   - Describe how you tested your solution in the **`Testing procedures`** section
   - Describe any challenges you faced and how you overcame them in the **`Challenges`** section.
   - Add complete attribution of all sources you used including documentation, source code, libraries, coding examples, etc., in the **`Full attribution of sources`** section.
   - If you implemented any extra credit features, you must describe them fully in the **`Extra credit`** section.

     > **Note**

> If you add any extra credit features and don't tell me about them here,
> I won't know about them, resulting in no extra credit for your extra work. 😞
> I won't be going through your code looking for them.

4. **Styling**: Use CSS to style your website including the overall page, the map, the list of selected points, the markers, the user input window, etc., to make them visually appealing. A trivial treatment of styling will not earn full points. Do **not** use Tailwind, UnoCSS, Bootstrap, Bulma, shadcn/ui, or other CSS builders or libraries.
5. **Responsiveness**: Ensure that your website is responsive and works well on both desktop and mobile devices.
6. **Resilience**: No programming error messages are ever exposed to the user, and only to the console log if it is important. Similarly, nothing the user can do will crash your code. Actions like the user entering a huge amount of text describing a dropped marker location or the user repeatedly hitting **Reset**, or any other unkind behaviour 😈 should not cause any problems.
7. **Code Quality**: Write clean, well-organized, and commented code to make it easy to understand and maintain. Choose *either* camelCase 🐫 or snake_case 🐍 for variables and functions. Remember: if I can't understand it, I can't grade it. 😖

   The `git` commit messages you write are importan too. They should be meaningful and precise. This is part of building large software systems on a team, and is part of code quality.

   > **Note**
   >
   > Any code you use that was created by another entity (human or AI ✨) must be acknowledged. Be sure you understand it and can explain it to me if asked. If you cannot explain it, don't use it.

8. **Deployment**: Deploy your website on `render.com` and add the `onrender.com` URL to your `README.md`.
9. **Submission**: Regularly do `git commit` and `git push` of your work to insure against loss. Use branches to avoid stepping on previously completed work before you're ready to merge them. When finished, run your last `git commit` and a `git push` to send your work back to the repository on GitHub Classroom. After the due date/time, I will grade the contents of the `main` branch of that repository.

   > **⚠ Warning**
   >
   > Mid-Term grades are due shortly after this lab's due date. A late submission could result in a zero score for the Mid-Term grade calculation.

Extra Credit additions

You can earn up to a maximum of 4 points extra credit **IF AND ONLY IF** (1) your score for the base requirements is at least 25/30, and (2) you fully implement one or more of the following features:

1. Allow the user to edit or delete a location after it has been added to the map.
2. Utilize some free API that provides additional information about the locations (e.g., city name, weather, population, etc.) and display that information when a marker is clicked.
3. Allow the user to save the list of locations to a file (e.g., JSON or CSV) and load it back into the map.

Additional functionality is encouraged **as long as** it is documented - otherwise it might confuse me. However, any such additions will not count as extra credit.

Resources

- Leaflet.js (a popular library for interactive maps)
- React leaflet examples by Grzegorz Tomicki
- OpenStreetMap API Documentation
- MDN *JavaScript frameworks and libraries* tutorial
- React Documentation